

React Router

What is React Router?

React Router is a standard library for routing in React applications. It allows you to handle navigation, map URLs to components, and manage browser history—all in a single-page application (SPA) context.

Enables client-side routing.

Keeps the UI in sync with the URL.

Supports nested routes, dynamic routes, and route guards.

Install it → `npm install react-router-dom`

React Router provides different routers based on how you want URLs to be handled:

Router	How it works	Use case
BrowserRouter	Uses HTML5 history API (pushState, popState)	Most modern apps
HashRouter	Uses # in URLs (example.com/#/home)	Apps hosted on static file servers without server-side routing

You have to Wrap app inside BrowserRouter to use routing

```
import { BrowserRouter } from "react-router-dom";
```

```
function App() {  
  return (  
    <BrowserRouter>
```

```
<Routes>

  <Route path="/" element={<Home />} />

</Routes>

</BrowserRouter>

);

}
```

Routes & Route

Routes is a container for all route definitions. Each Route maps a URL path to a React component.

Basic Example:

```
import { Routes, Route } from "react-router-dom";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
  );
}
```

Notes:

- path defines the URL.
- element is the component to render.

Nested Routes

React Router allows nesting, which is key for layouts:

```
<Routes>
<Route path="/dashboard" element={<DashboardLayout />}>
  <Route index element={<DashboardHome />} />
  <Route path="profile" element={<Profile />} />
  <Route path="settings" element={<Settings />} />
</Route>
</Routes>
```

in Dashboard <Outlet />

Notice:

- "profile" and "settings" are **relative paths**
- <Outlet /> renders the child route

If you forget <Outlet />, nothing renders.

URL /dashboard/profile will render Profile inside Dashboard.

d) Dynamic Routes (Route Parameters)

Dynamic segments allow URLs to pass variables:

```
<Route path="/user/:userId" element={<UserProfile />} />
```

In the component:

```
import { useParams } from "react-router-dom";
```

```
function UserProfile() {
  const { userId } = useParams();
  return <div>User ID: {userId}</div>;
}
```

- /user/123 → User ID: 123

Index Routes

When you want a default route for nested routes:

```

<Routes>
  <Route path="/dashboard" element={<Dashboard />}>
    <Route index element={<DashboardHome />} />
    <Route path="profile" element={<Profile />} />
  </Route>
</Routes>

```

- URL /dashboard → DashboardHome
- URL /dashboard/profile → Profile

Navigation

Instead of <a>, use <Link> to prevent full page reloads:

```
import { Link } from "react-router-dom";
```

```

function Navbar() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>
    </nav>
  );
}

```

Programmatic navigation:

```
import { useNavigate } from "react-router-dom";
```

```

function LoginButton() {
  const navigate = useNavigate();

  const handleLogin = () => {
    // login logic
    navigate("/dashboard"); // redirect after login
  };
}

```

```
return <button onClick={handleLogin}>Login</button>;  
}
```

h) 404 / Catch-All Routes

Use * to catch unmatched routes:

```
<Route path="*" element={<NotFound />} />
```

Relative vs Absolute Paths

- **Absolute path:** /about → always from root
- **Relative path:** profile → relative to parent route

Nested routes often use relative paths for cleaner URLs.

useLocation & useMatch

- `useLocation()` → get current URL, state, or search params.
- `useMatch(path)` → check if current URL matches a specific pattern.

```
const location = useLocation();  
console.log(location.pathname); // current path
```

Lazy Loading Routes

Use `React.lazy` + `Suspense` to split bundles:

```
const Dashboard = React.lazy(() => import("./Dashboard"));
```

```
<Routes>  
  <Route  
    path="/dashboard"  
    element={  
      <Suspense fallback={<div>Loading...</div>}>  
        <Dashboard />  
      </Suspense>  
    }  
  />  
</Routes>
```

```
    </Suspense>
  }
/>
</Routes>
```

3. Best Practices

1. **Use BrowserRouter for modern apps** and HashRouter only for static hosting.
2. **Keep routes centralized** in a separate routes.js file.
3. **Use nested routes** for layout reuse.
4. **Protect routes** with PrivateRoute or role-based guards.
5. **Lazy-load heavy components** to optimize performance.
6. **Use relative paths in nested routes** to avoid hardcoding URLs.
7. **Use <Outlet>** for rendering nested routes instead of duplicating layouts.

Q. What is the difference between <Link> and <a>?

<Link>

<a>

Client-side navigation Full page reload

Keeps React state Resets state

Uses history API Makes new HTTP request

👉 Always say: “We use <Link> to prevent full page reload in SPA.”

How do you pass data between routes?

```
navigate("/profile", { state: { user } });
```

Access:

```
const location = useLocation();
```

```
location.state
```